`maxentcpp`: A C++ reimplementation of Maximum Entropy species distribution modeling for R.

Ángel Luis Robles Fernández[1,*]

1 Department of Ecology and Evolutionary Biology, University of Kansas, Lawrence, KS, United States

*** a.l.robles.fernandez@gmail.com**
ORCID: 0000-0002-4741-8012

Abstract

`maxentcpp` is an R package offering a native C++17 reimplementation of the Maximum Entropy (Maxent) algorithm for species distribution modeling (SDM). The package translates the core continuous-predictor Maxent 3.4.4 fitting and projection algorithm into C++17 with R bindings via Rcpp, eliminating the Java Runtime Environment dependency entirely. It supports all six feature types of Java Maxent, including categorical predictors via `BinaryFeature`, output transformations in raw, logistic, and cloglog scales, a streaming raster evaluation engine for large-raster projections, and the full 1.0.0 diagnostic suite: jackknife variable importance, replicate runs and stratified cross-validation, and missing-data handling. Numerical fidelity is validated per-iteration against the original Java implementation via a companion test package.

Summary

Maximum Entropy (Maxent) modeling is the *de facto* standard for presence-only species distribution modeling (SDM) [8, 24, 27], with foundational papers cited tens of thousands of times across ecology, conservation biology, and biogeography. Yet the reference implementation remains a Java desktop application [23], and every species run in a typical workflow — e.g., 500 species × 4 climatic scenarios — spawns a JVM and round-trips data through SWD files on disk. `maxentcpp` is an R package that removes this friction with a native C++17 reimplementation of the Maxent algorithm. It estimates the geographic distribution of a species from presence-only occurrence records and environmental covariates by identifying the probability distribution of maximum entropy [16] subject to constraints derived from training data, in memory, with no Java runtime and no file serialization.

The fitted model is a *Gibbs distribution* over geographic space:

$$P(\mathbf{x}) = \frac{1}{Z} \exp\left(\sum_{j=1}^{J} \lambda_j f_j(\mathbf{x}) \right),$$

where f_j are feature transformations of the environmental variables, λ_j are learned weights, and $Z = \sum_{\mathbf{x}} \exp\big(\sum_j \lambda_j f_j(\mathbf{x})\big)$ is the normalizing constant (partition function). The optimizer finds the λ_j that maximize the regularized log-likelihood via sequential coordinate ascent with ℓ_1 (lasso) penalties [8, 24].

The original Maxent software is a Java desktop application [23]. `maxentcpp` translates the core continuous-predictor Maxent 3.4.4 fitting and projection algorithm into C++17 with R bindings via Rcpp [6] and RcppEigen [1], eliminating the Java Runtime Environment (JRE) dependency entirely. The package supports all six feature types of Java Maxent (linear, quadratic, product, hinge, threshold, and categorical via `BinaryFeature`), output transformations in raw, logistic, and complementary log-log (cloglog) scales, and a streaming raster evaluation engine that processes environmental layers block-by-block through the `terra` package [14], enabling large-raster projections without loading entire raster stacks into memory. Tools for diagnosing model performance include AUC evaluation, permutation importance, response curves, jackknife variable-importance analysis, replicate runs and stratified cross-validation, missing-data handling, and Multivariate Environmental Similarity Surfaces (MESS) [7] for novelty detection.

Statement of need

Maxent is the *de facto* standard for presence-only species distribution modeling [20]. However, the original Java implementation presents a number of practical barriers for present-day ecological research workflows:

1. **Java dependency.** Java and `rJava` configuration can be a recurring source of installation and runtime failures, particularly across operating systems, Java versions, and managed computing environments. While `conda` environments and Docker containers can manage Java versioning, they add deployment complexity and are not standard practice in many ecology research groups.

2. **File-based I/O.** Data must be serialized to disk as SWD or ASCII grid files before the Java process can read them, and results must be parsed back from output files. There is no in-memory bridge between R data structures and the Java optimizer.

3. **Scalability.** The Java implementation is single-threaded and GUI-centric. In multi-species, multi-scenario workflows (e.g., 500 species $\times$ 4 climatic scenarios), file I/O overhead accumulates: each species run requires writing SWD files, invoking the JVM, and parsing output files. A native in-memory API removes this per-call overhead by keeping data and model state in R objects.

4. **Maintenance.** The Java Maxent codebase has received only minor updates since version 3.4.4, with limited active development [23].

`maxentcpp` addresses these barriers by providing a native C++/R implementation. At present, the development version can be installed from GitHub using `remotes::install_github("alrobles/maxentcpp")`; following CRAN acceptance, the package will be installable with `install.packages("maxentcpp")`. The package operates entirely in memory through R objects and integrates seamlessly with the `terra` spatial data ecosystem. The intended users are ecologists, conservation biologists, and biogeographers who employ Maxent in reproducible, script-based workflows — from individual species analyses to large-scale biodiversity assessments.

Related work

`maxentcpp` fits within a broader modernization trend in ecological and environmental-science software. Circuitscape.jl [12] illustrates how established ecological

algorithms can be reimplemented in a modern high-performance language to improve scalability and parallel execution, while retaining continuity with a widely used landscape-connectivity tool. In R, the spatial ecosystem has similarly moved from older `sp`/`raster`-centered workflows toward `sf` [21] and `terra` [14], and SDM packages such as `ENMeval` [17] and `biomod2` [31] have followed this transition — with `ENMeval` additionally removing Java/`rJava` from its default path in favor of `maxnet`.

Within Maxent workflows specifically, existing modernization efforts address different parts of the problem. **dismo** [15] is the most widely used R interface to Java Maxent; it wraps `maxent.jar` via `rJava`, inheriting the JDK dependency and file I/O overhead, and is currently in maintenance mode following the transition from `raster` to `terra`. **rmaxent** [2] provides Java-free projection of previously fitted Maxent models from `.lambdas` files, parses feature weights and normalizers, and implements MESS-related diagnostics, but does not replace Java for model fitting. **maxnet** [10, 22] provides a complete Java-free fitting implementation based on `glmnet` coordinate descent — preserving the same statistical model but employing a different optimization algorithm that can produce numerically different results in some settings. **ENMeval** [17] is a model tuning and evaluation framework that wraps either `dismo::maxent()` or `maxnet::maxnet()`; it does not implement the Maxent algorithm itself. **kuenm** [3] is a calibration and evaluation toolkit that relies on `dismo` or direct Java Maxent calls.

`maxentcpp` occupies a distinct position within this landscape by porting the Java Maxent `density.Sequential` training optimizer itself to C++17 and validating optimizer trajectories against the Java implementation. To the best of current knowledge, `maxentcpp` is the first R package to target the Java Maxent 3.4.4 `density.Sequential` optimizer through a native C++ port and to publish per-iteration trajectory tests via the companion package `maxentcppCompTest` [9]. Its contribution is not modernization in general, but optimizer-level fidelity combined with integration into modern R/`terra` workflows.

Ecosystem comparison

The R ecosystem contains several packages that provide access to MaxEnt models, each employing a distinct integration strategy:

Package	MaxEnt integration	Dependency
dismo [15]	rJava bridge to `maxent.jar`	Java JDK, rJava
kuenm [3]	`system2("java -jar maxent.jar")`	Java JDK
kuenm2 (Cobos et al.)	`glmnet` via forked `maxnet` code	glmnet
wallace [18]	Delegates to ENMeval → maxnet or dismo	ENMeval
ENMTools [36]	`dismo::maxent()` directly	dismo, rJava
biomod2 [31]	Java (`system2`) or `maxnet`	Optional Java or maxnet
rmaxent [2]	Java-free projection of fitted Maxent models, `.lambdas` parsing, MESS	None (projection only)
MIAmaxent [34]	Maximum entropy via subset selection (not lasso); decoupled transformation/fitting/selection	glmnet
SDMtune [33]	Trains/tunes SDMs via `dismo::maxent()` or `maxnet`; hyperparameter tuning, variable selection	dismo or maxnet
maxentcpp	Native C++17 via Rcpp	None (self-contained)

These packages can be grouped by their relationship to the MaxEnt algorithm:

- **Black-box wrappers** (dismo, kuenm, ENMTools, biomod2 MAXENT mode): invoke the Java binary and read output files. The optimizer is inaccessible.
- **Approximate reimplementations** (maxnet, kuenm2, biomod2 MAXNET mode): use `glmnet` coordinate descent on the logistic regression formulation. The statistical model is equivalent but the optimization path differs, which can produce numerically different results in some settings.
- **Faithful reimplementation** (maxentcpp): ports the original Sequential optimizer to C++ and proves per-iteration numerical equivalence.

The distinguishing contribution of `maxentcpp` is that it is the only package offering a compiled native reimplementation of the actual MaxEnt `density.Sequential` optimizer — preserving both the statistical model and the optimization algorithm while eliminating the Java dependency.

Empirical comparison: maxentcpp vs maxnet

To quantify the practical differences between `maxentcpp` and `maxnet`, we compared both packages on a virtual species [19] with known true habitat suitability, constructed from the environmental layers bundled with `maxentcpp` (Annual Mean Temperature and Annual Precipitation over Central America, 2,371 non-NA cells). The virtual species has a Gaussian niche centered at 20 °C and 1,500 mm, and 100 presence records were sampled proportionally to the true suitability surface (see the companion vignette Virtual Species Comparison for full reproducible code).

Both packages were fitted with linear, quadratic, and hinge features. In this illustrative example, the two packages produced highly similar rank and spatial-overlap patterns:

Metric	Value
Schoener's D [30, 35]	0.93
Spearman ρ (rank correlation)	0.95
Pearson r	0.97
Mean $\lvert\Delta\text{cloglog}\rvert$	0.053
Max $\lvert\Delta\text{cloglog}\rvert$	0.43

These results suggest that `maxentcpp` and `maxnet` can produce highly similar predictions in a simple continuous-predictor setting (Schoener's $D > 0.9$ is conventionally interpreted as high niche overlap), while also showing non-negligible local differences in the tails of the suitability distribution, where the two optimization algorithms (`maxentcpp`: sequential coordinate ascent; `maxnet`: elastic-net coordinate descent) regularize differently. A broader simulation study across multiple species, sample sizes, and predictor correlation structures would be needed to establish general equivalence.

The key practical scenarios where a user must choose `maxentcpp` over `maxnet` are: (1) when exact reproduction of Java Maxent results is required (e.g., replicating published analyses), (2) when lambda file interoperability with Java Maxent is needed, and (3) when streaming raster projection onto grids larger than available RAM is required.

Performance benchmarks

Training time for the bundled *Abeillia abeillei* dataset (73 presences, 2,371 background, linear + quadratic + hinge features):

Package	Median time per fit
`maxentcpp`	~820 ms
`maxnet`	~530 ms

`maxnet` is faster for small datasets because `glmnet`'s coordinate descent is highly optimized for dense feature matrices. `maxentcpp`'s streaming evaluation avoids materializing the full prediction matrix, which is expected to reduce peak memory during projection; future benchmarks should quantify this advantage on rasters larger than available RAM.

To exercise the 1.0.0 feature set, we benchmarked the new diagnostics on a synthetic grid of 2,371 cells and 100 presence records with two continuous predictors plus one five-level categorical variable (linear + quadratic + hinge features; `max_iter = 500`):

1.0.0 feature	Median wall time
Single fit (continuous only)	~1 ms
Fit with categorical predictor	~94 ms
Jackknife (3 variables; 6 fits)	~75 s
Cross-validation ($k = 5$; 5 fits)	~84 s
Replicate runs (bootstrap, $n = 5$; 5 fits)	~108 s

Each 1.0.0 diagnostic is a small multiple of single fits, as expected: jackknife runs one fit per variable per leave-out scheme, and cross-validation and replicate runs each fit k or n models. These wall times reflect the full per-fit cost on a mid-size grid and are linear in the number of constituent fits; on the smaller bundled *Abeillia abeillei* dataset they are correspondingly faster. Full reproducible code is provided in the package vignettes.

Software design

Architecture

`maxentcpp` was built as a new package rather than contributing to existing projects for three reasons. First, a faithful port of the original Java optimizer (sequential coordinate ascent using ℓ_1-penalized Newton steps) requires a C++ engine that cannot be expressed through `glmnet`'s API; contributing this to `maxnet` would change its core algorithm. Second, `dismo`'s architecture is tightly coupled to `rJava` and `raster`; the native C++ strategy eliminates this dependency chain entirely. Third, the header-based C++ library design of `maxentcpp` enables reuse in other packages or standalone applications beyond R — something not achievable through modifications to existing R-only packages.

`maxentcpp` is organized in three layers (C++ core, Rcpp bridge, R interface), with the C++ core further split into algorithmic and infrastructure sublayers:

Layer	Key components	Lines
C++ core (algorithmic)	`Sequential`, `FeaturedSpace`, six feature types	˜4,600
C++ core (I/O, diagnostics)	`BackgroundProvider`, grid I/O, CSV, MESS, response curves	˜2,300
Rcpp bridge	`rcpp_*.cpp` external-pointer bindings [5]	˜2,300
R interface	`maxent_run()`, feature generators, projection, evaluation, diagnostics, replicates, jackknife	˜5,000

The C++ core uses Eigen [11] for dense linear algebra. Of the ˜6,900 C++ lines, approximately 4,600 implement the core algorithmic logic (optimizer, features, density) while the remainder handles I/O and diagnostics. The high-level `maxent_run()` function provides a one-call workflow mirroring the Java GUI experience, while lower-level functions (`maxent_generate_features()`, `maxent_featured_space()`, `maxent_fit()`, `maxent_project_cloglog()`) offer full control over each modeling step.

Optimization algorithm

The `Sequential` optimizer is a direct port of `density.Sequential` in Java Maxent 3.4.4, as represented by the public `mrmaxent/Maxent` repository and AMNH release notes (where 3.4.4 is described as a minor bug-fix release). It minimizes the ℓ_1-regularized negative log-likelihood:

$$\mathcal{L}(\boldsymbol{\lambda}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{J}\lambda_j f_j(\mathbf{x}_i) + \log Z(\boldsymbol{\lambda}) + \sum_{j=1}^{J}\beta_j|\lambda_j|,$$

where m is the number of presence records, $Z(\boldsymbol{\lambda}) = \sum_{k=1}^{N}\exp\big(\sum_j \lambda_j f_j(\mathbf{x}_k)\big)$ is the partition function summed over N background points, and $\beta_j = \hat{\sigma}_j/\sqrt{m}$ is the per-feature regularization parameter with $\hat{\sigma}_j$ being the standard deviation of feature j over the presence points (floored at 0.001).

At each iteration, the optimizer selects the feature j^* with the most negative $\Delta\mathcal{L}$ bound (the `deltaLossBound` function; block-coordinate ascent [32]), then computes a Newton step:

$$\alpha_j = -\frac{\partial\mathcal{L}/\partial\lambda_j}{\mathbf{u}_j^\top \mathbf{H}\mathbf{u}_j},$$

where $\mathbf{u}_j$ is the j-th coordinate direction and $\mathbf{u}_j^\top\mathbf{H}\mathbf{u}_j = \mathrm{Var}_q[f_j]$ is the variance of feature j under the current distribution q. The derivative includes the ℓ_1 subgradient: $\partial\mathcal{L}/\partial\lambda_j = \mathbb{E}_q[f_j] - \mathbb{E}_{\hat{p}}[f_j] + \beta_j\,\mathrm{sign}(\lambda_j)$. The step is damped during early iterations ($\alpha \leftarrow \alpha/50$ for iterations < 10, $\alpha/10$ for < 20, $\alpha/3$ for < 50) and falls back to a line search (`searchAlpha`) if the Newton step does not decrease loss. Every 10 iterations, a parallel update applies coordinate steps to all features simultaneously.

Convergence is tested every 20 iterations: training stops when the relative loss improvement falls below 10^{-5} or 500 iterations are reached. All constants in this section are taken directly from Java Maxent 3.4.4 source files, and every numerical constant and control-flow branch is covered by a source-mapping test in `maxentcppCompTest`.

Streaming raster evaluation

`maxentcpp` reads `terra::SpatRaster` objects block-by-block through a `BackgroundProvider` abstraction. Each tile is scored by the C++ engine and discarded

before the next tile is loaded, allowing projection onto raster stacks larger than available RAM. The partition function Z is accumulated across tiles by summing unnormalized densities $\exp(\sum_j \lambda_j f_j(\mathbf{x}_k))$ for each cell k in the tile, then normalizing once after all tiles are processed. This produces results equivalent to single-pass evaluation because the mathematical sum is commutative and each cell is scored independently; floating-point summation order may introduce differences at the least-significant bit level.

Numerical fidelity

Each C++ class is a direct translation of its Java counterpart, preserving the same control flow, regularization logic, and numerical constants:

`maxentcpp` (C++)	Java Maxent original
`LinearFeature`	`density.LinearFeature`
`QuadraticFeature`	`density.SquareFeature`
`ProductFeature`	`density.ProductFeature`
`HingeFeature`	`density.HingeFeature`
`ThresholdFeature`	`density.ThresholdFeature`
`BinaryFeature`	`density.BinaryFeature`
`FeaturedSpace`	`density.FeaturedSpace`
`Sequential::run()`	`density.Sequential.run()`

This design choice prioritizes backward compatibility: users migrating from Java Maxent should obtain numerically equivalent results for implemented features under matched inputs, defaults, and deterministic settings. The fidelity contract is enforced by a dedicated companion package, `maxentcppCompTest` [9], which runs the C++ optimizer and the original Java `density.Sequential` on shared test fixtures and compares per-iteration trajectories of loss, entropy, and λ vectors.

On controlled test fixtures (identical input data, same floating-point representation), agreement reaches $< 10^{-14}$ relative error for symmetric fixtures and $< 10^{-6}$ on $\|\Delta\lambda\|_\infty$ for asymmetric fixtures at every iteration checkpoint. **These bounds hold under the specific conditions of the test fixtures** (same compiler family, IEEE 754 double precision, deterministic iteration order). Floating-point ordering differences introduced by alternative BLAS backends or aggressive compiler optimizations (e.g., `-ffast-math`) could widen the gap, though the sequential nature of the coordinate-ascent algorithm limits sensitivity to parallelism-induced reordering. Cross-platform CI (Ubuntu, macOS, Windows with GCC, Clang, and MSVC) confirms that the test fixtures pass on all three compiler families.

Lambda file compatibility. Trained models are serialized in the same `.lambdas` text format used by Java Maxent 3.4.4. Lambda files produced by either implementation can be loaded by the other, ensuring interoperability with existing workflows and archived models.

Feature completeness

The following table summarizes the current scope of `maxentcpp` relative to Java Maxent 3.4.4 and `maxnet`:

Legend: ✓ = implemented and tested; ∘ = partial or experimental; — = not implemented.

Feature	`maxentcpp`	Java Maxent 3.4.4	`maxnet` 0.1.4
Linear features	✓	✓	✓
Quadratic features	✓	✓	✓
Product features	✓	✓	✓
Hinge features	✓	✓	✓
Threshold features	✓	✓	✓
Raw/logistic/cloglog output	✓	✓	✓
AUC evaluation	✓	✓	via ENMeval
Permutation importance	✓	✓	via ENMeval
Response curves	✓	✓	∘
MESS maps	✓	✓	—
Clamping / fade-by-clamping [25]	✓	✓	—
Lambda file I/O	✓	✓	—
Streaming raster projection	✓	—	—
Bias grids	✓	✓	via `offset`
Categorical variables	✓	✓	✓
Replicate runs / cross-validation	✓	✓	via ENMeval
Jackknife variable selection	✓	✓	—
Missing data handling	✓	✓	✓
SWD-to-raster projection	—	✓	—

Categorical variables, replicate runs, jackknife, missing-data handling, and background bias weighting are now implemented and tested. SWD-to-raster projection remains a workflow convenience not yet exposed through the public R API; users can still project models by loading environmental grids via `terra` and the package's grid conversion functions.

Development provenance

The translation followed a phased approach across four publicly accessible repositories:

1. `alrobles/Maxent` — a fork of the original `mrmaxent/Maxent` [23] Java source code, where the architecture was analyzed and each Java class was mapped to a C++ target.
2. `alrobles/maxentcpp-devel` — the development repository where the C++ port, Rcpp bindings, R interface, tests, and documentation were iteratively built and refined.
3. `alrobles/maxentcppCompTest` — the cross-language fidelity test suite containing Java oracle runners, golden trajectory files, and automated comparison scripts.
4. `alrobles/maxentcpp` — the release repository with the clean, CRAN-ready package.

Limitations and inherited assumptions

The Maxent 3.4.4 algorithm that `maxentcpp` faithfully reproduces has well-documented limitations that users should be aware of:

- **Feature explosion.** The number of features (particularly hinge and threshold) grows with the number of environmental variables, which can cause identifiability issues in high-dimensional settings [8, 20].

- **Sensitivity to background sampling.** Maxent's output is influenced by the choice and size of the background sample, which affects the estimated density and can bias predictions in undersampled regions [26].
- ℓ_1 **regularization limitations.** The sequential coordinate ascent with ℓ_1 penalties produces sparse models but does not guarantee selection of the "correct" features under high correlation among predictors [13].
- **No principled uncertainty quantification.** Unlike Bayesian approaches or bootstrap-based methods, the Maxent optimizer produces point estimates without confidence intervals.

A principled alternative would be to reformulate the Maxent objective using modern convex optimization tooling (e.g., proximal gradient methods, ADMM) or Bayesian inference. However, such reformulations would produce different results than the widely used Java implementation, breaking comparability with published analyses. The design of `maxentcpp` explicitly aims to preserve this comparability.

CRAN compliance and software quality

Achieving CRAN acceptance requires more than passing `R CMD check`: packages must meet strict software-quality standards that safeguard user environments and ensure reproducibility across platforms [4, 29]. During the pre-submission review, `maxentcpp` was brought to compliance on four fronts. First, all 36 example blocks were migrated from `\dontrun{}` to `\donttest{}` [28] and rewritten to be self-contained with synthetic data, so every documented example is live, testable code that CRAN's infrastructure can execute with `--run-donttest`. Second, functions that touch global graphics state now restore it through an internal `.maxent_safe_png()` helper built on `on.exit()`, guaranteeing cleanup even on error [29]. Third, all `terra`-dependent examples are guarded with `requireNamespace("terra", quietly = TRUE)` so they skip gracefully when the optional dependency is unavailable. Fourth, GitHub Actions runs `R CMD check --as-cran` on Ubuntu (R release and R-devel), macOS, and Windows, plus a CRAN-preflight job with `_R_CHECK_FORCE_SUGGESTS_=false` that simulates CRAN's minimal-dependency environment.

Research impact statement

`maxentcpp` provides a numerically faithful implementation of the core continuous-predictor Maxent fitting and projection workflow for the implemented feature classes and output transformations, including categorical predictors, replicate runs and cross-validation, jackknife diagnostics, and missing-data handling. SWD-to-raster projection remains a convenience workflow not yet exposed through the public R API.

The package is distributed under the MIT license (compatible with Eigen's MPL2 and RcppEigen's GPL-2 via the "or later" clause), with full source code, a pkgdown documentation website at `https://alrobles.github.io/maxentcpp/`, and four vignettes covering a getting-started walkthrough, the mathematical foundations of Maxent features, a comparative motivation document, and a virtual species validation study. Continuous integration via GitHub Actions runs `R CMD check` on Ubuntu, macOS, and Windows across R release and R-devel. The test suite comprises over 20 test files (˜4,800 lines) covering feature generation, optimization convergence, spatial projection, Java numerical equivalency, model diagnostics, and end-to-end workflows.

By providing a dependency-free, memory-efficient, and numerically faithful Maxent implementation, `maxentcpp` may lower the barrier for large-scale biodiversity

assessments, climate-change projections, and conservation prioritization studies that rely on presence-only species distribution models, bringing the package to near-parity with Java Maxent 3.4.4 for the implemented feature classes while remaining installable in Java-free environments.

AI usage disclosure

Generative AI tools were used during the development of `maxentcpp` and the preparation of this manuscript, as detailed below.

Tools used.

- **GitHub Copilot** (GPT-4-based, 2025 version): code scaffolding and boilerplate generation during the initial porting phases in `alrobles/Maxent` and `alrobles/maxentcpp-devel`.
- **Devin** (Cognition AI, 2025–2026): incremental feature implementation, test writing, documentation generation, CRAN compliance fixes, CI configuration, and manuscript drafting.
- **Claude** (Anthropic, 2025–2026): complex refactoring, cross-cutting documentation, and code review.

Nature and scope of assistance. AI tools contributed to: C++ code generation and refactoring from Java source, Rcpp binding scaffolding, R wrapper functions, `testthat` test scaffolding and expansion, `roxygen2` documentation, vignette drafting, pkgdown site configuration, CI workflow setup, CRAN compliance review, and manuscript drafting. The core algorithmic C++ classes (`Sequential`, `FeaturedSpace`, feature types) were translated from Java with AI assistance but under direct human supervision: each class was mapped line-by-line from the corresponding Java source, reviewed for correctness against the Java reference, and validated through the `maxentcppCompTest` trajectory comparison framework. Approximately 60% of the C++ algorithmic lines were initially drafted by AI tools and subsequently reviewed and corrected by the human author; the remaining 40% were written directly by the author, particularly the performance-critical optimizer inner loops and numerical edge cases.

Maintainability. The human author (Á. L. Robles Fernández) can independently explain and modify every algorithmically significant class. As evidence: the `Sequential::run()` optimizer, `good_alpha()`, `delta_loss_bound()`, `newton_step_feature()`, and `do_parallel_update()` methods are documented with line-by-line references to the corresponding Java source (e.g., "mirrors Sequential.java:294..342"), and the author has independently debugged numerical discrepancies during the fidelity testing process.

Confirmation of review. All AI-generated code, tests, documentation, and manuscript text were reviewed, edited, and validated by the human author, who made all core design decisions including the algorithm translation strategy, API design, numerical fidelity requirements, and the phased development architecture. The full development history is publicly available in the four repositories listed above.

Acknowledgements

The author thanks Steven J. Phillips, Robert P. Anderson, and Robert E. Schapire for creating the original Maxent software and making the source code publicly available under an MIT license. The `Rcpp`, `RcppEigen`, and `terra` package maintainers are gratefully acknowledged for providing the infrastructure that makes `maxentcpp` possible.

References

1. D. Bates and D. Eddelbuettel. Fast and elegant numerical linear algebra using the RcppEigen package. *Journal of Statistical Software*, 52(5):1–24, 2013.

2. J. Baumgartner. rmaxent: Tools for working with Maxent in R, 2017.

3. M. E. Cobos, A. T. Peterson, N. Barve, and L. Osorio-Olvera. kuenm: an R package for detailed development of ecological niche models using Maxent. *PeerJ*, 7:e6281, 2019.

4. CRAN Repository Maintainers. CRAN repository policy, 2024. Revision 6846.

5. D. Eddelbuettel. *Seamless R and C++ Integration with Rcpp*. Springer, New York, 2013.

6. D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011.

7. J. Elith, M. Kearney, and S. Phillips. The art of modelling range-shifting species. *Methods in Ecology and Evolution*, 1:330–342, 2010.

8. J. Elith, S. J. Phillips, T. Hastie, M. Dudík, Y. E. Chee, and C. J. Yates. A statistical explanation of MaxEnt for ecologists. *Diversity and Distributions*, 17(1):43–57, 2011.

9. Á. L. R. Fernández. maxentcppCompTest: Cross-language fidelity tests for maxentcpp, 2025.

10. J. Friedman, T. Hastie, and R. Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010.

11. G. Guennebaud, B. Jacob, et al. Eigen v3, 2010.

12. K. R. Hall, R. Anantharaman, V. A. Landau, M. Clark, B. G. Dickson, A. Jones, J. Platt, A. Edelman, and V. B. Shah. Circuitscape in Julia: Empowering dynamic approaches to connectivity assessment. *Land*, 10(3):301, 2021.

13. T. Hastie, R. Tibshirani, and M. Wainwright. *Statistical Learning with Sparsity: The Lasso and Generalizations*. CRC Press, 2015.

14. R. J. Hijmans. *terra: Spatial Data Analysis*, 2024. R package version 1.7-78.

15. R. J. Hijmans, S. Phillips, J. Leathwick, and J. Elith. *dismo: Species Distribution Modeling*, 2023. R package version 1.3-14.

16. E. T. Jaynes. Information theory and statistical mechanics. *Physical Review*, 106:620–630, 1957.

17. J. M. Kass, R. Muscarella, P. J. Galante, C. L. Bohl, G. E. Pinilla-Buitrago, R. A. Boria, M. Soley-Guardia, and R. P. Anderson. ENMeval 2.0: Redesigned for customizable and reproducible modeling of species' niches and distributions. *Methods in Ecology and Evolution*, 12(9):1602–1608, 2021.

18. J. M. Kass, B. Vilela, M. E. Aiello-Lammens, R. Muscarella, C. Merow, and R. P. Anderson. wallace: A flexible platform for reproducible modeling of species niches and distributions built for community expansion. *Methods in Ecology and Evolution*, 9(4):1151–1156, 2018.

19. B. Leroy, R. Delsol, B. Hugueny, C. N. Meynard, C. Baroni, et al. virtualspecies, an R package to generate virtual species distributions. *Ecography*, 39(6):599–607, 2016.

20. C. Merow, M. J. Smith, and J. A. Silander, Jr. A practical guide to MaxEnt for modeling species' distributions: what it does, and why inputs and settings matter. *Ecography*, 36(10):1058–1069, 2013.

21. E. Pebesma. Simple features for R: Standardized support for spatial vector data. *The R Journal*, 10(1):439–446, 2018.

22. S. J. Phillips. *maxnet: Fitting 'Maxent' Species Distribution Models with 'glmnet'*, 2024. R package version 0.1.4.

23. S. J. Phillips, R. P. Anderson, M. Dudík, R. E. Schapire, and M. E. Blair. Opening the black box: an open-source release of Maxent. *Ecography*, 40(7):887–893, 2017.

24. S. J. Phillips, R. P. Anderson, and R. E. Schapire. Maximum entropy modeling of species geographic distributions. *Ecological Modelling*, 190(3–4):231–259, 2006.

25. S. J. Phillips and M. Dudík. Modeling of species distributions with Maxent: new extensions and a comprehensive evaluation. *Ecography*, 31:161–175, 2008.

26. S. J. Phillips and M. Dudík. Modeling of species distributions with Maxent: new extensions and a comprehensive evaluation. *Ecography*, 31(2):161–175, 2008.

27. S. J. Phillips, M. Dudík, and R. E. Schapire. A maximum entropy approach to species distribution modeling. In *Proceedings of the Twenty-First International Conference on Machine Learning*, ICML '04, pages 655–662, New York, NY, USA, 2004. ACM.

28. R Contributors. CRAN cookbook: General issues, 2024.

29. R Core Team. *Writing R Extensions*, 2024. Version 4.4.0.

30. T. W. Schoener. The Anolis lizards of Bimini: resource partitioning in a complex fauna. *Ecology*, 49:704–726, 1968.

31. W. Thuiller, D. Georges, and R. Engler. biomod2: Ensemble platform for species distribution modeling. *R package*, 2009.

32. P. Tseng. Convergence of a block coordinate descent method for nondifferentiable minimization. *Journal of Optimization Theory and Applications*, 109:475–494, 2001.

33. S. Vignali, A. G. Barras, R. Arlettaz, and V. Braunisch. SDMtune: An R package to tune and evaluate species distribution models. *Ecology and Evolution*, 10(20):11488–11506, 2020.

34. J. Vollering, R. Halvorsen, and I. G. Alsos. Measuring niche breadth and overlap with random modelling iterations: MIAmaxent. *Ecology and Evolution*, 9(23):13674–13687, 2019.

35. D. L. Warren, R. E. Glor, and M. Turelli. Environmental niche equivalency versus conservatism: quantitative approaches to niche evolution. *Evolution*, 62:2868–2883, 2008.

36. D. L. Warren, R. E. Glor, and M. Turelli. ENMTools: a toolbox for comparative studies of environmental niche models. *Ecography*, 33(3):607–611, 2010.